Document number: P1968R0
Date: 2019-11-07
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2017
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "tentatively ready" Issues for the November, 2019 (Belfast) meeting

References in this document reflect the section and paragraph numbering of document WG21 N4835.

1621. Member initializers in anonymous unions

Section: 11.10.2 [class.base.init] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 2013-02-12

The effect of a non-static data member initializer in an anonymous union is not clearly described in the current wording. Consider the following example:

```
struct A {
  struct B {
    union {
      int x = 37;
    };
    union {
      int y = x + 47;  // Well-formed?
    };
  } a;
};
```

Does an anonymous union have a constructor that applies a non-static data member initializer? Or is the initialization performed by the constructor of the class in which the anonymous union appears? In particular, is the reference to `x` in the initializer for `y` well-formed or not? If the initialization of `y` is performed by `B`'s constructor, there is no problem because `B::x` is a member of the object being initialized. If an anonymous union has its own constructor, `B::x` is just a member of the containing class and is a reference to a non-static data member without an object, which is ill-formed. Implementations currently appear to take the latter interpretation and report an error for that initializer.

As a further example, consider:

```
union {       // #1
  union {     // #2
    union {   // #3
      int y = 32;
    };
  };
} a { } ;
```

One interpretation might be that union #3 has a non-trivial default constructor because of the initializer of `y`, which would give union #2 a deleted default constructor, which would make the example ill-formed.

As yet another example, consider:

```
union {
  union {
    int x;
  };
  union {
    int y = 3;
  };
  union {
    int z;
  };
} a { };
```

Assuming the current proposed resolution of issue 1502, what is the correct interpretation of this code? Is it well-formed, and if so, what initialization is performed?

Finally, consider

```
struct S {
  union { int x = 1; };
  union { int y = 2; };
} s{};
```

Does this violate the prohibition of aggregates containing member initializers in 9.4.1 [dcl.init.aggr] paragraph 1?

See also issues 1460, 1562, 1587, and 1623.

Proposed resolution (October, 2019):

1. Change 11.4.2 [class.mfct.non-static] paragraph 1 as follows:

> ...A non-static member function may also be called directly using the function call syntax (7.6.1.2 [expr.call], 12.4.1.1 [over.match.call]) from within ~~the body of a member function of~~ its class or ~~of~~ a class derived from its class**, or a member thereof, as described below.** .

2. Change 11.4.2 [class.mfct.non-static] paragraph 3 as follows:

> When an *id-expression* (7.5.4 [expr.prim.id]) that is not part of a class member access syntax (7.6.1.4 [expr.ref]) and not used to form a pointer to member (7.6.2.1 [expr.unary.op]) is used in a member of class `X` in a context where `this` can be used (7.5.2 [expr.prim.this]), if name lookup (6.5 [basic.lookup]) resolves the name in the *id-expression* to a non-static non-type member of some class `C`, and if either the *id-expression* is potentially evaluated or `C` is `X` or a base class of `X`, the *id-expression* is transformed into a class member access expression (7.6.1.4 [expr.ref]) using `(*this)` (11.4.2.1 [class.this]) as the *postfix-expression* to the left of the `.` operator. [*Note:* If `C` is not `X` or a base class of `X`, the class member access expression is ill-formed. —*end note*] ~~Similarly during name lookup, when an *unqualified-id* (7.5.4.1 [expr.prim.id.unqual]) used in the definition of a member function for class `X` resolves to a static member, an enumerator or a nested type of class `X` or of a base class of `X`, the *unqualified-id* is transformed into a *qualified-id* (7.5.4.2 [expr.prim.id.qual]) in which the *nested-name-specifier* names the class of the member function. These transformations do~~ **This transformation does** not apply in the template definition context (13.8.2.1 [temp.dep.type]). [*Example:*...

3. Delete 11.4.8 [class.static] paragraph 3:

> ~~If an unqualified-id (7.5.4.1 [expr.prim.id.unqual]) is used in the definition of a static member following the member's *declarator-id*, and name lookup (6.5.1 [basic.lookup.unqual]) finds that the *unqualified-id* refers to a static member, enumerator, or nested type of the member's class (or of a base class of the member's class), the *unqualified-id* is transformed into a *qualified-id* expression in which the *nested-name-specifier* names the class scope from which the member is referenced. [*Note:* See 7.5.4 [expr.prim.id] for restrictions on the use of non-static data members and non-static member functions. —*end note*]~~

4. Change 11.5.1 [class.union.anon] paragraph 1 as follows:

> A union of the form
>
> `union {` *member-specification* `} ;`
>
> is called an anonymous union; it defines an unnamed type and an unnamed object of that type called an anonymous union object. Each *member-declaration* in the *member-specification* of an anonymous union shall either define a non-static data member or be a static_*assert-declaration*. ~~[*Note:*~~ Nested types, anonymous unions, and functions ~~cannot~~ **shall not** be declared within an anonymous union. ~~—*end note*]~~ The names of the members...

2126. Lifetime-extended temporaries in constant expressions

Section: 7.7 [expr.const] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-05-20

Consider an example like the following:

```
typedef const int CI[3];
constexpr CI &ci = CI{11, 22, 33};
static_assert(ci[1] == 22, "");
```

This is ill-formed because the lifetime of the array temporary did not start within the current evaluation. Perhaps we should treat all lifetime-extended temporaries of const-qualified literal type that are initialized by constant expressions as if they are `constexpr` objects?

Proposed resolution (October, 2019):

Change 7.7 [expr.const] paragraph 3 as follows:

> A variable is *usable in constant expressions* after its initializing declaration is encountered if it is a constexpr variable, or it is a constant-initialized variable of reference type or of const-qualified integral or enumeration type. An object or reference is *usable in constant expressions* if it is
>
> - a variable that is usable in constant expressions, or
> - a template parameter object (13.2 [temp.param]), or
> - a string literal object (5.13.5 [lex.string]), or
> - **a temporary object of non-volatile const-qualified literal type whose lifetime is extended (6.7.7 [class.temporary]) to that of a variable that is usable in constant expressions, or**

- a non-mutable subobject or reference member of any of the above~~, or~~**.**
- ~~a complete temporary object of non-volatile const-qualified integral or enumeration type that is initialized with a constant expression.~~

This resolution also resolves issue 2439.

2282. Consistency with mismatched aligned/non-over-aligned allocation/deallocation functions

Section: 7.6.2.7 [expr.new] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-06-27

The fallback treatment for alignment and non-alignment allocation and deallocation functions is asymmetric. While a deletion of a non-overaligned class object will match a class-specific alignment deallocation function if no class-specific non-alignment deallocation function is provided, the same is not true for allocation: a *new-expression* for a non-overaligned class type will fail if an alignment allocation function is provided with no non-alignment allocation function. The allocation behavior should be changed to match the deallocation behavior.

Proposed resolution (September, 2019):

Change 7.6.2.7 [expr.new] paragraph 18 as follows, splitting the running text into bullets:

> Overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. If the type of the allocated object has new-extended alignment, the next argument is the type's alignment, and has type `std::align_val_t`. If the *new-placement* syntax is used, the *initializer-clause*s in its *expression-list* are the succeeding arguments. If no matching function is found **then**
>
> - ~~and~~ **if** the allocated object type has new-extended alignment, the alignment argument is removed from the argument list~~,~~**;**
> - **otherwise, an argument list that is the type's alignment and has type `std::align_val_t` is added into the argument list immediately after the first argument;**
>
> and **then** overload resolution is performed again.

2347. Passing short scoped enumerations to ellipsis

Section: 7.6.1.2 [expr.call] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2017-04-28

According to 7.6.1.2 [expr.call] paragraph 9,

> If the argument has integral or enumeration type that is subject to the integral promotions (7.3.6 [conv.prom]), or a floating-point type that is subject to the floating-point promotion (7.3.7 [conv.fpprom]), the value of the argument is converted to the promoted type before the call. These promotions are referred to as the default argument promotions.

A scoped enumeration with an underlying type that is shorter than `int` will not be widened when passed to an ellipsis. Should it be?

Notes from the June, 2018 meeting:

The consensus of CWG was that the value passed ougnt to be widened to match the promoted type of the underlying type.

Proposed resolution (May, 2019): [SUPERSEDED]

Change 7.6.1.2 [expr.call] paragraph 12 as follows:

> ...If the argument has **an** integral or enumeration type that is subject to the integral promotions (7.3.6 [conv.prom]), **a scoped enumeration type whose underlying type is subject to the integral promotions,** or a floating-point type that is subject to the floating-point promotion (7.3.7 [conv.fpprom]), the value of the argument is converted to the promoted type before the call. These promotions are referred to as the *default argument promotions*.

Notes from the September, 2019 teleconference:

The consensus was that passing scoped enumerations to ellipsis should be conditionally-supported behavior, similar to the treatment of class types with nontrivial copy semantics.

Proposed resolution (October, 2019):

Change 7.6.1.2 [expr.call] paragraph 12 as follows:

> ...Passing a potentially-evaluated argument of **a scoped enumeration type or of a** class type (Clause 11 [class]) having an eligible non-trivial copy constructor, an eligible non-trivial move constructor, or a non-trivial destructor (11.4.3 [special]), with no corresponding parameter, is conditionally-supported with implementation-defined semantics. If the argument...

2374. Overly permissive specification of `enum` direct-list-initialization

Section: 9.4.4 [dcl.init.list] **Status:** tentatively ready **Submitter:** Shafik Yaghmour **Date:** 2018-02-18

According to 9.4.4 [dcl.init.list] bullet 3.8,

- Otherwise, if `T` is an enumeration with a fixed underlying type (9.7.1 [dcl.enum]), the *initializer-list* has a single element `v`, and the initialization is direct-list-initialization, the object is initialized with the value `T(v)` (7.6.1.3 [expr.type.conv]); if a narrowing conversion is required to convert `v` to the underlying type of `T`, the program is ill-formed.

The conversion `T(v)` is too broad, allowing, e.g., conversion from a different scoped enumeration type. The intent presumably was only to allow `v` to be a value of `T`'s underlying type.

Notes from the October, 2018 teleconference:

CWG agreed with the suggested direction, along the lines of "...can be implicitly converted to the underlying type of `T`..."

Proposed resolution (May, 2019): [SUPERSEDED]

Change bullet 3.8 of 9.4.4 [dcl.init.list] as follows:

- Otherwise, if `T` is an enumeration with a fixed underlying type (9.7.1 [dcl.enum]), the *initializer-list* has a single element `v`, **`v` can be implicitly converted to the underlying type of `T`,** and the initialization is direct-list-initialization, the object is initialized with the value `T(v)` (7.6.1.3 [expr.type.conv]); if a narrowing conversion is required to convert `v` to the underlying type of `T`, the program is ill-formed. [*Example:*...

Proposed resolution (October, 2019):

Change bullet 3.8 of 9.4.4 [dcl.init.list] as follows:

- Otherwise, if `T` is an enumeration with a fixed underlying type (9.7.1 [dcl.enum]) **`U`**, the *initializer-list* has a single element `v`, **`v` can be implicitly converted to `U`,** and the initialization is direct-list-initialization, the object is initialized with the value `T(v)` (7.6.1.3 [expr.type.conv]); if a narrowing conversion is required to convert `v` to ~~the underlying type of `T`~~ **`U`**, the program is ill-formed. [*Example:*...

2399. Unclear referent of "expression" in *assignment-expression*

Section: 7.6.19 [expr.ass] **Status:** tentatively ready **Submitter:** Lisa Lippincott **Date:** 2019-02-13

According to 7.6.19 [expr.ass] paragraph 3,

> If the left operand is not of class type, the expression is implicitly converted (7.3 [conv]) to the cv-unqualified type of the left operand.

Since the second operand of an assignment operator can now be an *initializer-clause*, the referent of "expression" is unclear.

See also issue 1542.

Proposed resolution (May, 2019): [SUPERSEDED]

Change 7.6.19 [expr.ass] paragraph 3 as follows:

> If the left operand is not of class type **and the right operand is an *assignment-expression***, the ~~expression~~ ***assignment-expression*** is implicitly converted (7.3 [conv]) to the cv-unqualified type of the left operand.

Proposed resolution (October, 2019):

Change 7.6.19 [expr.ass] paragraph 3 as follows:

> ~~The expression~~ **If the right operand is an expression, it** is implicitly converted (7.3 [conv]) to the cv-unqualified type of the left operand.

2419. Loss of generality treating pointers to objects as one-element arrays

Section: 7.6.6 [expr.add] **Status:** tentatively ready **Submitter:** Andrey Erokhin **Date:** 2019-07-15

CCCC,

Before the resolution of issue 1596, 7.6.6 [expr.add] specified that:

> For the purposes of these operators, a pointer to a nonarray object behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type.

where “these operators” refers to the additive operators. This provision thus applied to any pointer, regardless of its provenance. In its place, the normative provision for this treatment was restricted to the `&` operator only, in 7.6.2.1 [expr.unary.op] paragraph 3:

> For purposes of pointer arithmetic (7.6.6 [expr.add]) and comparison (7.6.9 [expr.rel], 7.6.10 [expr.eq]), an object that is not an array element whose address is taken in this way is considered to belong to an array with one element of type `T`.

Thus, for example:

```
int *p1 = new int;
int *p2 = &*p1;
bool b1 = p1 < p1+1;   // undefined behavior
bool b2 = p2 < p2+1;   // well-defined
```

This restriction does not seem desirable.

Proposed resolution (October, 2019):

1. Change 7.6.2.1 [expr.unary.op] bullet 3.2 as follows:

   > The result of the unary `&` operator is a pointer to its operand.
   >
   > - ...
   > - Otherwise, if the operand is an lvalue of type `T`, the resulting expression is a prvalue of type “pointer to `T`” whose result is a pointer to the designated object (6.7.1 [intro.memory]) or function. [*Note:* In particular, taking the address of a variable of type “*cv* `T`” yields a pointer of type “pointer to *cv* `T`”. —*end note*] ~~For purposes of pointer arithmetic (7.6.6 [expr.add]) and comparison (7.6.9 [expr.rel], 7.6.10 [expr.eq]), an object that is not an array element whose address is taken in this way is considered to belong to an array with one element of type `T`.~~
   > - ...

2. Change 6.8.2 [basic.compound] paragraph 3 as follows:

   > ...A value of a pointer type that is a pointer to or past the end of an object *represents the address* of the first byte in memory (6.7.1 [intro.memory]) occupied by the object[43] or the first byte in memory after the end of the storage occupied by the object, respectively. [*Note:* A pointer past the end of an object (7.6.6 [expr.add]) is not considered to point to an unrelated object of the object's type that might be located at that address. A pointer value becomes invalid when the storage it denotes reaches the end of its storage duration; see 6.7.5 [basic.stc]. —*end note*] For purposes of pointer arithmetic (7.6.6 [expr.add]) and comparison (7.6.9 [expr.rel], 7.6.10 [expr.eq]), a pointer past the end of the last element of an array `x` of *n* elements is considered to be equivalent to a pointer to a hypothetical element `x[`*n*`]` **and an object of type `T` that is not an array element is considered to belong to an array with one element of type `T`**. The value representation of pointer types...

3. Change the footnote in 7.6.6 [expr.add] bullet 4.2 as follows:

When an expression `J` that has integral type is added to or subtracted from an expression `P` of pointer type, the result has the type of `P`.

- ...
- Otherwise, if `P` points to an array element *i* of an array object `x` with *n* elements (9.3.3.4 [dcl.array]), [*Footnote:* ~~An~~ **As specified in 6.8.2 [basic.compound]. an** object that is not an array element is considered to belong to a single-element array for this purpose~~; see 7.6.2.1 [expr.unary.op]. A~~ **and a** pointer past the last element of an array `x` of *n* elements is considered to be equivalent to a pointer to a hypothetical array element *n* for this purpose~~; see 6.8.2 [basic.compound]~~. —*end footnote*] the expressions...

4. Change the footnote in 7.6.9 [expr.rel] paragraph 4 aa follows:

> The result of comparing unequal pointers to objects [*Footnote:* ~~An~~ **As specified in 6.8.2 [basic.compound], an** object that is not an array element is considered to belong to a single-element array for this purpose~~; see 7.6.2.1 [expr.unary.op]. A~~ **and a** pointer past the last element of an array `x` of *n* elements is considered to be equivalent to a pointer to a hypothetical **array** element ~~`x [` *n* `]`~~ ***n*** for this purpose~~; see 6.8.2 [basic.compound]~~. —*end footnote*] is defined in terms of a partial order consistent with the following rules:

5. Change 25.9.15 [numeric.ops.midpoint] paragraph 5 as follows:

> *Expects:* `a` and `b` point to, respectively, elements `x[`*i*`]` and `x[`*j*`]` of the same array object `x`. [*Note:* ~~An~~ **As specified in 6.8.2 [basic.compound], an** object that is not an array element is considered to belong to a single-element array for this purpose~~; see 7.6.2.1 [expr.unary.op]. A~~ **and a** pointer past the last element of an array `x` of *n* elements is considered to be equivalent to a pointer to a hypothetical **array** element ~~`x[`*n*`]`~~ ***n*** for this purpose~~; see 6.8.2 [basic.compound]~~. —*end note*]

2422. Incorrect grammar for *deduction-guide*

Section: 13.11 [temp.deduct.guide] **Status:** tentatively ready **Submitter:** Barry Revzin **Date:** 2019-07-28

According to 13.11 [temp.deduct.guide] paragraph 1, the syntax of a *deduction-guide* is:

> *deduction-guide:*
> `explicit`$_{opt}$ *template-name* `(` *parameter-declaration-clause* `)` `->` *simple-template-id* `;`

Instead of `explicit`, this production should use the *explicit-specifier* nonterminal. (The wording of 12.4.1.8 [over.match.class.deduct] has references to the *explicit-specifier* of a deduction guide, for example.)

Proposed resolution (September, 2019):

Change the grammar in 13.11 [temp.deduct.guide] paragraph 1 as follows:

> *deduction-guide:*
> ~~`explicit`~~ ***explicit-specifier***$_{opt}$ *template-name* `(` *parameter-declaration-clause* `)` `->` *simple-template-id* `;`

2424. `constexpr` initialization requirements for variant members

Section: 9.2.5 [dcl.constexpr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2019-08-03

Paper P1331R2 removed the requirement that a constexpr constructor initialize every non-variant non-static data member, but it left untouched the corresponding requirements for variant members. That is, the modified text in 9.2.5 [dcl.constexpr] paragraph 4 still contains:

> The definition of a constexpr constructor whose *function-body* is not `= delete` shall additionally satisfy the following requirements:
>
> - if the class is a union having variant members (11.5 [class.union]), exactly one of them shall be initialized;
> - if the class is a union-like class, but is not a union, for each of its anonymous union members having variant members, exactly one of them shall be initialized;

Presumably this was an oversight and these two bullets should be changed from “exactly” to “at most” or something similar.

Proposed resolution (September, 2019):

Delete the indcated text from 9.2.5 [dcl.constexpr] paragraph 4:

> The definition of a constexpr constructor whose *function-body* is not `= delete` shall additionally satisfy the following requirements:
>
> - ~~if the class is a union having variant members (11.5 [class.union]), exactly one of them shall be initialized;~~
> - ~~if the class is a union-like class, but is not a union, for each of its anonymous union members having variant members, exactly one of them shall be initialized;~~
> - for a non-delegating constructor, every constructor selected to initialize non-static data members and base class subobjects shall be a constexpr constructor;
> - for a delegating constructor, the target constructor shall be a constexpr constructor.

2426. Reference to destructor that cannot be invoked

Section: 14.3 [except.ctor] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2019-08-13

Consider the following example:

```
template<typename T> struct A {
private:
  ~A() {}
};
```

```
A<char> g();
A<char> f() { return g(); }
```

According to 14.3 [except.ctor] paragraph 2,

> If an exception is thrown during the destruction of temporaries or local variables for a `return` statement (8.7.3 [stmt.return]), the destructor for the returned object (if any) is also invoked.

In `f()` there is no possibility of an exception occuring during the processing of the `return` statement, so there appears to be no reason for a reference to the private destructor of `A<char>`. Current implementations, however, issue an access error for this example. Is wording needed to make that destructor potentially invoked in such cases?

Proposed resolution (September, 2019):

1. Change 8.7.3 [stmt.return] paragraph 2 as follows:

> ...[*Note:* A `return` statement can involve an invocation of a constructor to perform a copy or move of the operand if it is not a prvalue or if its type differs from the return type of the function. A copy operation associated with a `return` statement may be elided or converted to a move operation if an automatic storage duration variable is returned (11.10.5 [class.copy.elision]). —*end note*] [*Example:*
>
> ```
> std::pair<std::string,int> f(const char* p, int x) {
>   return {p,x};
> }
> ```
>
> —*end example*] **The destructor for the returned object is potentially invoked (11.4.6 [class.dtor], 14.3 [except.ctor]). [*Example:***
>
> ```
> class A {
>   ~A() {}
> };
> A f() { return A(); }    // error: destructor of A is private (even though it is never invoked)
> ```
>
> ***—end example*]** Flowing off the end...

2. Change 11.4.6 [class.dtor] paragraph 14 as follows:

> A destructor can also be invoked explicitly. A destructor is potentially invoked if it is invoked or as specified in 7.6.2.7 [expr.new], **8.7.3 [stmt.return],** 9.4.1 [dcl.init.aggr], 11.10.2 [class.base.init], and 14.2 [except.throw]. A program is ill-formed if a destructor that is potentially invoked is deleted or not accessible from the context of the invocation.

2427. Deprecation of volatile operands and unevaluated contexts

Section: 7.6.19 [expr.ass] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2019-08-14

According to 7.6.19 [expr.ass] paragraph 7,

> A simple assignment whose left operand is of a volatile -qualified non-class type is deprecated (D.5 [depr.volatile.type]) unless the assignment is either a discarded-value expression or appears in an unevaluated context.

The deprecations of increment, decrement, and compound assignment operators do not, but presumably should, mention unevaluated contexts.

Notes from the August, 2019 teleconference:

The omission of those operators was intentional; the deprecation is intended only to affect cases where using the result of the operation would result in a subsequent fetch of the value. However, some shortcomings of the existing wording were noted and will be addressed in the resolution.

Proposed resolution (October, 2019):

Change 7.6.19 [expr.ass] paragraph 5 as follows:

> A simple assignment whose left operand is of a `volatile`-qualified type is deprecated (D.5 [depr.volatile.type]) unless the **(possibly parenthesized)** assignment is ~~either~~ a discarded-value expression or ~~appears in~~ an unevaluated ~~context~~ **operand**.

2429. Initialization of `thread_local` variables referenced by lambdas

Section: 8.8 [stmt.dcl] **Status:** tentatively ready **Submitter:** Princeton Ferro **Date:** 2019-08-22

According to 6.7.5.2 [basic.stc.thread] paragraph 2,

> A variable with thread storage duration shall be initialized before its first odr-use (6.3 [basic.def.odr]) and, if constructed, shall be destroyed on thread exit.

According to 8.8 [stmt.dcl] paragraph 4, for block-scope variables this initialization is peformed when the declaration statement is executed:

> Dynamic initialization of a block-scope variable with static storage duration (6.7.5.1 [basic.stc.static]) or thread storage duration (6.7.5.2 [basic.stc.thread]) is performed the first time control passes through its declaration; such a variable is considered initialized upon the completion of its initialization.

However, there are cases in which the flow of control does not pass through a variable's declaration, leaving it uninitialized in spite of it being odr-used. For example:

```
#include <thread>
#include <iostream>

struct Object {
  int i;
  Object() : i(3) {}
};

int main(void) {
  static thread_local Object o;

  std::cout << "[main] o.i = " << o.i << std::endl;
  std::thread([] {
    std::cout << "[new thread] o.i = " << o.i << std::endl;
  }).join();
}
```

`o` is a block-scope variable with thread storage and a dynamic initializer. The lambda passed into `std::thread`'s constructor refers to `o` but does not capture it, which should be fine since `o` is not a variable with automatic storage. However, when control passes through the lambda, it will do so on a new thread. When that happens, it will refer to `o` for the first time. Because `o` is a thread-local variable, it should be initialized, but because it is declared in block scope, it will only be initialized when control passes through its declaration, which will never happen on the new thread.

This example is straightforward, but others are more ambiguous:

```
#include <thread>
#include <iostream>

struct Object {
  int i;
  Object(int v) : i(3 + v) {}
};

int main(void) {
  int w = 4;
  static thread_local Object o(w);

  std::cout << "[main] o.i = " << o.i << std::endl;
  std::thread([] {
    std::cout << "[new thread] o.i = " << o.i << std::endl;
  }).join();
}
```

Here the initialization of `o` uses the value of `w`, which is not captured. Perhaps it should be ill-formed for a lambda to refer to a block-scope thread-local variable.

Proposed resolution (October, 2019):

Change 6.7.5.2 [basic.stc.thread] paragraphs 1 and 2 as follows:

> All variables declared with the `thread_local` keyword have *thread storage duration*. The storage for these entities ~~shall last~~ **lasts** for the duration of the thread in which they are created. There is a distinct object or reference per thread, and use of the declared name refers to the entity associated with the current thread.
>
> **[*Note:*** A variable with thread storage duration ~~shall be~~ **is** initialized ~~before its first odr-use (6.3 [basic.def.odr])~~ **as specified in 6.9.3.2 [basic.start.static], 6.9.3.3 [basic.start.dynamic], and 8.8 [stmt.dcl]** and, if constructed, ~~shall be~~ **is** destroyed on thread exit **(6.9.3.4 [basic.start.term])**. ~~—~~ ***end note*****]**

2430. Completeness of return and parameter types of member functions

Section: 11.4 [class.mem] **Status:** tentatively ready **Submitter:** Krystian Stasiowski **Date:** 2019-08-23

According to 11.4 [class.mem] paragraph 7,

> A class is considered a completely-defined object type (6.8 [basic.types]) (or complete type) at the closing } of the *class-specifier*. The class is regarded as complete within its complete-class contexts; otherwise it is regarded as incomplete within its own class *member-specification*.

The complete-class contexts (paragraph 6) include the body of a member function but not its return and parameter types. Thus it appears that an example like the following is ill-formed:

```
struct S {
  S f(S s) { return s; }
};
```

because of 9.5.1 [dcl.fct.def.general] paragraph 2

> The type of a parameter or the return type for a function definition shall not be an incomplete or abstract (possibly cv-qualified) class type in the context of the function definition unless the function is deleted (9.5.3 [dcl.fct.def.delete]).

The words “in the context of the function definition” were added by the resolution of issue 1824 to address this problem, but “context” is most naturally read as referring to the lexical context where the definition appears rather than within its body.

Proposed resolution (October, 2019):

Change 9.5.1 [dcl.fct.def.general] paragraph 2 as follows:

> ...The type of a parameter or the return type for a function definition shall not be ~~an incomplete or abstract (possibly cv-qualified) class type in the context of the function definition~~ **a (possibly cv-qualified) class type that is incomplete or abstract within the function body** unless the function is deleted (9.5.3 [dcl.fct.def.delete]).

2431. Full-expressions and temporaries bound to references

Section: 6.9 [basic.exec] **Status:** tentatively ready **Submitter:** Andrey Erokhin **Date:** 2019-02-07

According to 6.9 [basic.exec] paragraph 5,

> A *full-expression* is
>
> - ...
> - an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7 [class.temporary]), or
> - ...

This definition excludes the destruction of temporaries that are bound to references from being treated as full-expressions. It is not clear whether this omission has observable effects or not. See editorial issue 2664.

Proposed resolution (October, 2019):

Change 6.9 [basic.exec] bullet 5.5 as follows:

> A *full-expression* is
>
> - ...

- an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.7.7 [class.temporary]) **whose lifetime has not been extended**, or
- ...

2432. Return types for defaulted <=>

Section: 11.11.3 [class.spaceship] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2019-08-14

It is unclear what the constraints are on the type R. We define the "synthesized three-way comparison for comparison category type R", but it's defined in such a way that it works for an arbitrary type R, and the uses of it do not impose a constraint that R is a comparison category type. Should it be permissible to default an `operator<=>` with some other return type, so long as the construction described in 11.11.3 [class.spaceship] works (specifically, so long as all subobjects have `operator<=>`s that can be converted to the specified return type)?

Proposed resolution (September, 2019)

Change 11.11.3 [class.spaceship] paragraphs 1-3, changing the running text of paragraph 2 into a bulleted list, as follows:

> The *synthesized three-way comparison* ~~for comparison category~~ **of** type `R` (17.11.2 [cmp.categories]) of glvalues `a` and `b`...
>
> **Let `R` be the declared return type of a defaulted three-way comparison operator function.** Given an expanded list of subobjects for an object `x` of type `C`, **let R_i be** the type of the expression x_i `<=>` x_i ~~is denoted by R_i. If~~ **, or `void` if** overload resolution ~~as~~ applied to ~~x_i `<=>` x_i~~ **that expression** does not find a usable function~~, then R_i is `void`~~.
>
> - If ~~the declared return type of a defaulted three-way comparison operator function~~ **`R`** is `auto`, then the return type is deduced as the common comparison type (see below) of R_0, R_1, ..., R_{n-1}. If the return type is deduced as `void`, the operator function is defined as deleted.
> - ~~If the declared return type of a defaulted three-way comparison operator function is `R` and~~ **Otherwise, if** the synthesized three-way comparison ~~for comparison category~~ **of** type `R` between any objects x_i and x_i is not defined or would be ill-formed, the operator function is defined as deleted.
>
> ...until the first index *i* where the synthesized three-way comparison ~~for comparison category~~ **of** type `R` between x_i and y_i yields...

2433. Variable templates in the ODR

Section: 6.3 [basic.def.odr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2019-10-10

The list of entities in 6.3 [basic.def.odr] paragraph 12 that can have multiple definitions across translation units does not, but should, include variable templates.

Proposed resolution (October, 2019):

> There can be more than one definition of a
>
> - class type (Clause 11 [class]),
> - enumeration type (9.7.1 [dcl.enum]),
> - inline function **or variable** ~~with external linkage~~ (9.2.7 [dcl.inline]),
> - ~~inline variable with external linkage (9.2.7 [dcl.inline]),~~
> - ~~class template (Clause 13 [temp]),~~
> - ~~non-static function template (13.7.6 [temp.fct]),~~
> - ~~concept (13.7.8 [temp.concept]),~~
> - ~~static data member of a class template (13.7.1.3 [temp.static]),~~
> - ~~member function of a class template (13.7.1.1 [temp.mem.func]),~~
> - ~~template specialization for which some template parameters are not specified (13.9 [temp.spec], 13.7.5 [temp.class.spec]),~~
> - **templated entity (13.1 [temp.pre]),**
> - default argument for a parameter (for a function in a given scope), or
> - default template argument
>
> in a program provided that...

[Drafting note: "with external linkage" is not needed for the `inline` *entities because the other cases - entities attached to a named module and multiple definitions in the same translation unit - are ruled out later in that paragraph.]*

2437. Conversion of `std::strong_ordering` in a defaulted `operator<=>`

Section: 11.11.3 [class.spaceship] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2019-08-14

According to 11.11.3 [class.spaceship] paragraph 3,

> The return value `V` of type `R` of the defaulted three-way comparison operator function with parameters `x` and `y` of the same type is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for `x` and `y` (in increasing index order) until the first index *i* where the synthesized three-way comparison for comparison category type `R` between x_i and y_i yields a result value v_i where v_i `!=` 0, contextually converted to `bool`, yields `true`; `V` is v_i converted to `R`. If no such index exists, `V` is `std::strong_ordering::equal` converted to `R`.

This is meaningless, however, because the kind of conversion is not specified. According to bullet 1.1,

- If overload resolution for `a <=> b` finds a usable function (12.4 [over.match]), `static_cast<R>(a <=> b)`.

so consistency would suggest that `static_cast<R>(std::strong_ordering::equal)` is probably the right answer.

Proposed resolution (October, 2019):

Change 11.11.3 [class.spaceship] paragraph 3 as follows:

> ...If no such index exists, `V` is **`static_cast<R>(`**`std::strong_ordering::equal`**`)`** ~~converted to `R`~~.

2439. Undefined term in definition of “usable in constant expressions”

Section: 7.7 [expr.const] **Status:** tentatively ready **Submitter:** Davis Herring **Date:** 2019-08-28

According to 7.7 [expr.const] paragraph 3,

> An object or reference is *usable in constant expressions* if it is
>
> - ...
> - a complete temporary object of non-volatile const-qualified integral or enumeration type that is initialized with a constant expression.

The phrase “initialized with a constant expression” is not defined (which causes problems with `std::is_constant_evaluated`). It would be better to use “is constant-initialized” instead.

Proposed resolution (October, 2019):

This issue is resolved by the resolution of issue 2126.

2442. Incorrect requirement for default arguments

Section: 12.4.2 [over.match.viable] **Status:** tentatively ready **Submitter:** S. B. Tam **Date:** 2019-09-24

(From editorial issue 3235.)

According to 12.4.2 [over.match.viable] bullet 2.3 says,

> First, to be a viable function, a candidate function shall have enough parameters to agree in number with the arguments in the list.
>
> - ...

- A candidate function having more than *m* parameters is viable only if the *(m+1)*st parameter has a default argument (9.3.3.6 [dcl.fct.default]). [*Footnote:* According to 9.3.3.6 [dcl.fct.default], parameters following the *(m+1)*st parameter must also have default arguments. —*end footnote*] For the purposes of overload resolution, the parameter list is truncated on the right, so that there are exactly *m* parameters.

However, this is incorrect; 9.3.3.6 [dcl.fct.default] paragraph 4 permits parameter packs to follow parameters with default arguments.

Proposed resolution (October, 2019):

Change 12.4.2 [over.match.viable] bullet 2.3 as follows:

- A candidate function having more than *m* parameters is viable only if ~~the *(m+1)*st parameter has a default argument (9.3.3.6 [dcl.fct.default]). [*Footnote:* According to 9.3.3.6 [dcl.fct.default], parameters following the *(m+1)*st parameter must also have default arguments. —*end footnote*]~~ **all parameters following the *m*th have default arguments (9.3.3.6 [dcl.fct.default]).** For the purposes of overload resolution, the parameter list is truncated on the right, so that there are exactly *m* parameters.